

清华大学电子工程系本科生核心课

数据与算法 - 第1讲

课程概述、数据、数学模型和算法

冯伟 (fengwei@tsinghua.edu.cn)

2024年9月10日





提纲

- 课程概述
- 数据
- 数学模型
- 算法

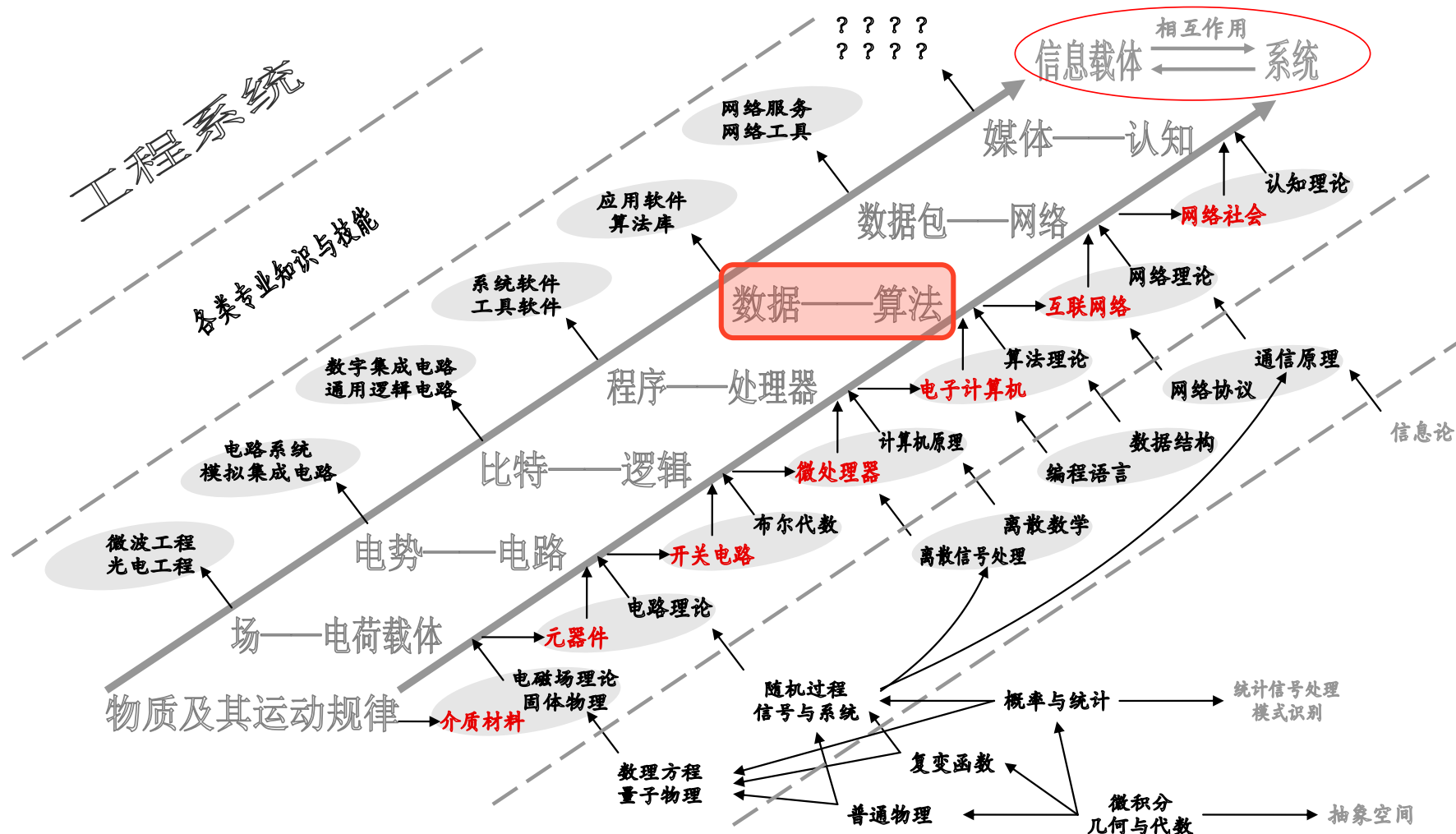


提纲

- 课程概述
- 数据
- 数学模型
- 算法

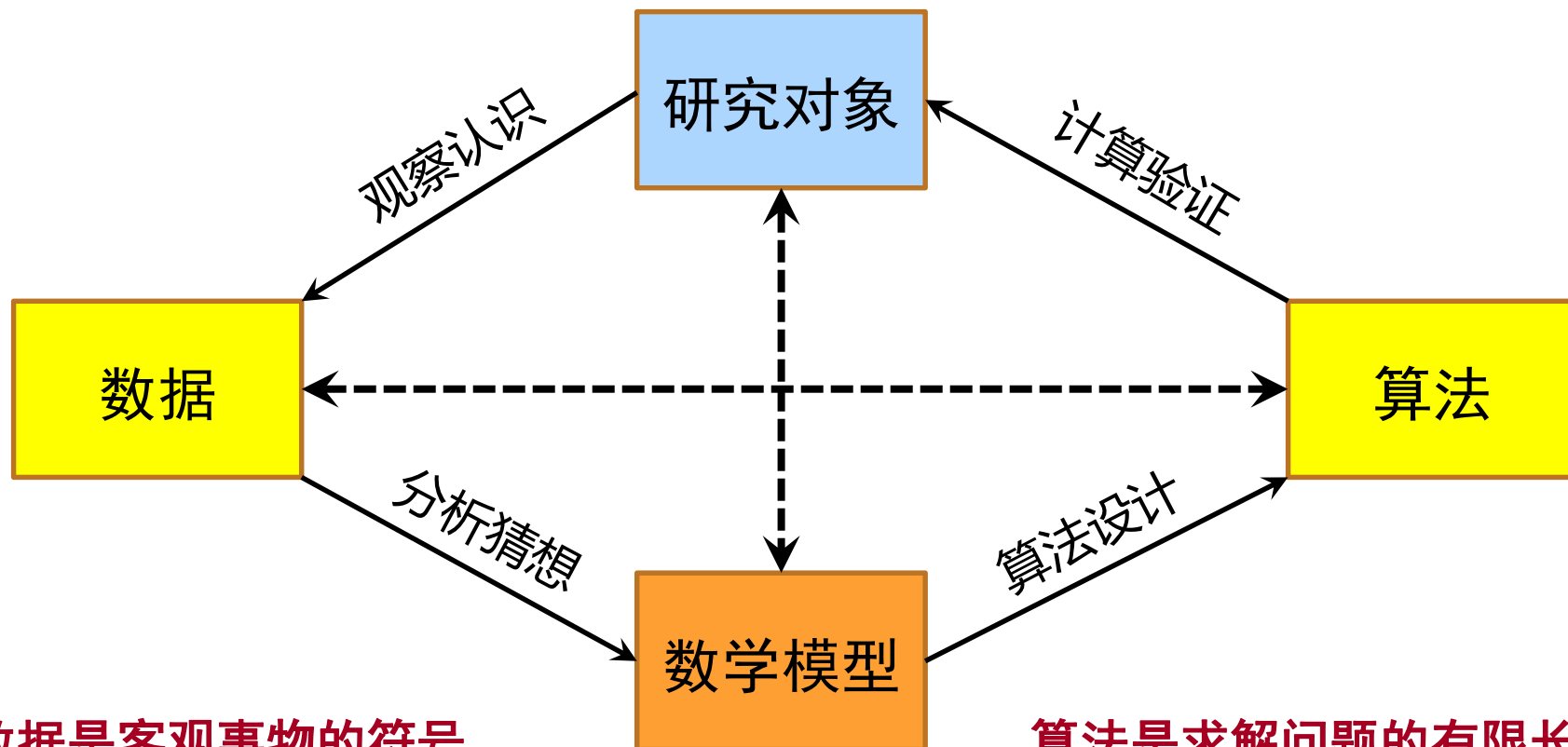


课程概述





主要内容



数据是客观事物的符号表示，是包含信息和知识的原始材料

算法是求解问题的有限长度的指令序列，是特定问题的程序化解决方案

信息抽取
知识发现



教学日历 (第2周, 9/17原排课程停上)

周次	日期	教学内容
1	9/10	课程概述、数据、数学模型和算法
3	9/24	二元关系、抽象数据类型、线性表
3	9/29	栈、递归、队列 (补第4周10月1日课程)
5	10/8	优先级队列、串、串匹配
6	10/15	树、二叉树 (一)
7	10/22	树、二叉树 (二)
8	10/29	图 (一)
9	11/5	图 (二)、查找
10	11/12	排序
11	11/19	数值计算概念、误差
12	11/26	线性方程组
13	12/3	拟合与插值
14	12/10	非线性方程、最优化初步
15	12/17	算法设计思想 (一)
16	12/24	算法设计思想 (二)、课程回顾



教材和参考书

■ 教材

- 《数据与算法》，吴及、陈健生、白铂，清华大学出版社

■ 参考书

- 《数据结构与算法》，朱明方、吴及，清华大学出版社
- 《数值方法与计算机实现》，徐士良，清华大学出版社
- 《The Art of Computer Programming》，Donald Ervin Knuth
- 《算法（第4版）》，Robert Sedgewick & Kevin Wayne



关于D. E. Knuth (唐纳德.E.克努特)



- 算法和程序设计技术的先驱者
- 《计算机程序设计的艺术》(The Art of Computer Programming)：作用与地位可与数学史上欧几里得的《几何原本》相比
- 荣获1974年图灵奖，这一年他只有36岁；至今仍保持图灵奖最年轻获奖者的纪录
- 计算机排版系统TeX和字型设计系统METAFONT的发明者



关于D. E. Knuth (唐纳德.E.克努特)



- 他认为一个计算机程序应该是雅致的、甚至可以说是美的。计算机程序设计应该是一门艺术，一个算法应该像一段音乐，而一个好的程序应该如一部文学作品一般
- 李开复：**练内功**，不要只花功夫学习各种流行的编程语言和工具，以及一些公司招聘广告上要求的科目。要把数据结构、算法、数据库、离散数学等基础课程学好



课程基本情况

- 课程名称：数据与算法
- 学分：3
- 时间：周二第2大节（09:50-12:15）
- 地点：六教 6A102
- **学时：48（15次课）**
- 先修课程：线性代数、微积分、离散数学、计算机程序设计基础
- **基础能力：C/C++编程**
- 任课教师：冯伟
- 任课教师主页：http://web.ee.tsinghua.edu.cn/fengwei/zh_CN/index.htm
- 联系方式：fengwei@tsinghua.edu.cn



课程微信群



课程基本情况

- 教学环节
 - 课堂授课 / 平时作业 (共5次) / 上机实验 (2~16周)
 - 闭卷考试
- 课件、作业等通过网络学堂发布
- 上机安排：周三15:00-18:30，周四11:30-22:00，周五15:00-18:30；中央主楼9楼教学实验中心 916 & 918；实验提交和判分通过OJ平台进行
- 成绩计算
 - 平时成绩：作业 (10%) + OJ实验 (40%)
 - 期末考试：闭卷 (50%)

别太纠结于此



答疑安排

地点：中央主楼9楼机房

总负责：张宇东，zhangyd16@mails.tsinghua.edu.cn

时间	助 教	电邮
周三15:00-18:30	刘天启	ltq22@mails.tsinghua.edu.cn
	常 锴	changk22@mails.tsinghua.edu.cn
周四11:30-15:00	郑凯俐	zkl22@mails.tsinghua.edu.cn
	王凯文	wangkaiwen521@sohu.com
周四15:00-18:30	沈震乾	szg22@mails.tsinghua.edu.cn
	刘光奕	895643049@qq.com
周四18:30-22:00	朱 洵	zhu-x24@mails.tsinghua.edu.cn
	尤心心	yxx23@mails.tsinghua.edu.cn
周五15:00-18:30	孙浩然	767780791shr@gmail.com
	姜 澈	jc23@mails.tsinghua.edu.cn



如何学好本课程

- **鱼**：概念（提前预习）、定性与定量
- **渔**：算法 → 方法（课上暂离电脑）
- **欲**：上进的欲望（扩展思考）
- **娱**：快乐学习
- **愚**：不走捷径（动手编程）
- **誉**：课程成绩 → 本科学业 → 人生发展



提纲

- 课程概述
- 数据
- 数学模型
- 算法

数据：无处不在

文字和符号



多媒体数据



通信信号



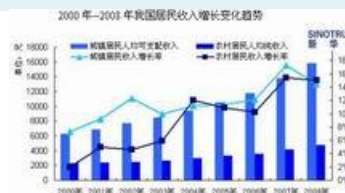
传感器数据



社会数据

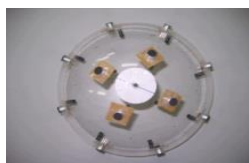
最新民调

民调机构	奥巴马	罗姆尼
盖洛普	47%	46%
CBS	46%	45%
CNN	49%	47%
ABC	46%	47%
FOX	44%	45%



数据：采集、传输、存储、处理

采集



传输



存储



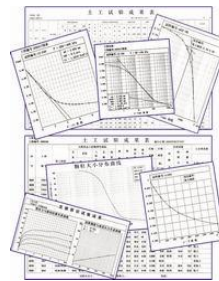
模拟信号



数字信号



处理





数据：采集、传输、存储、处理



相机水平、存储空间
逐渐超越通信，成为
手机的核心竞争指标！



变革

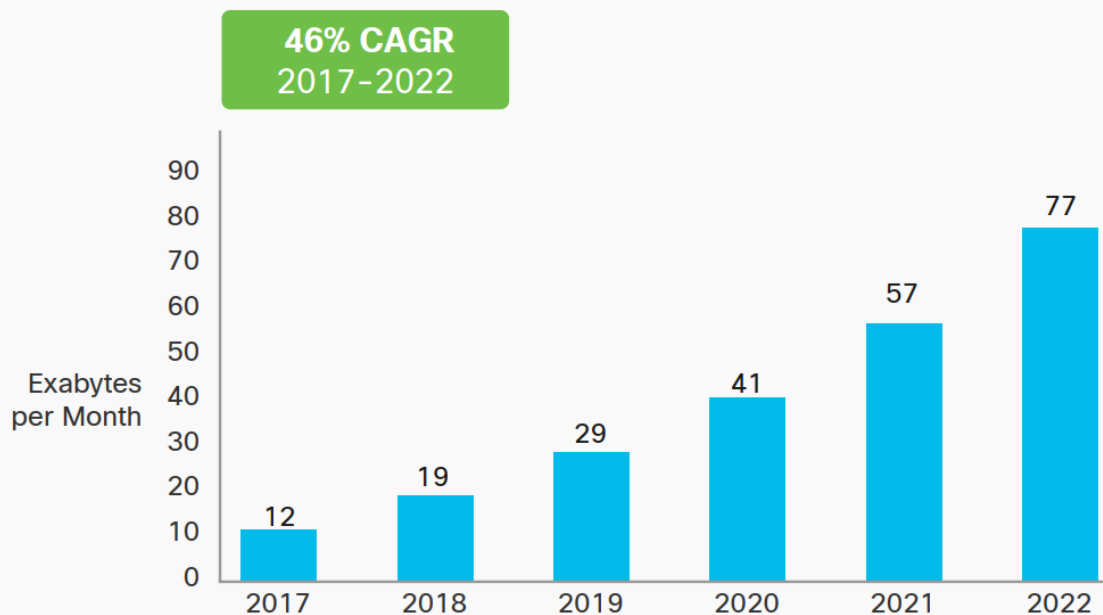


创新

大数据时代



Figure 2. Cisco Forecasts 77 Exabytes per Month of Mobile Data Traffic by 2022



Source: Cisco VNI Mobile, 2019

Byte(字节, 8bit) $\rightarrow$ KB(2^{10} Bytes) $\rightarrow$ MB(2^{20} Bytes) $\rightarrow$ GB(2^{30} Bytes) $\rightarrow$ TB(2^{40} Bytes)
 $\rightarrow$ PB(2^{50} Bytes) $\rightarrow$ EB(2^{60} Bytes) $\rightarrow$ ZB(2^{70} Bytes) $\rightarrow$ YB(2^{80} Bytes)

数据的用处： 例子 ①



第谷·布拉赫，丹麦天文学家和占星学家。在汶岛建造天堡观象台，经过20年的观测，积累了大量准确细致的天文观测数据。他编制的恒星表相当准确，至今仍然有使用价值，是近代天文学奠基人。

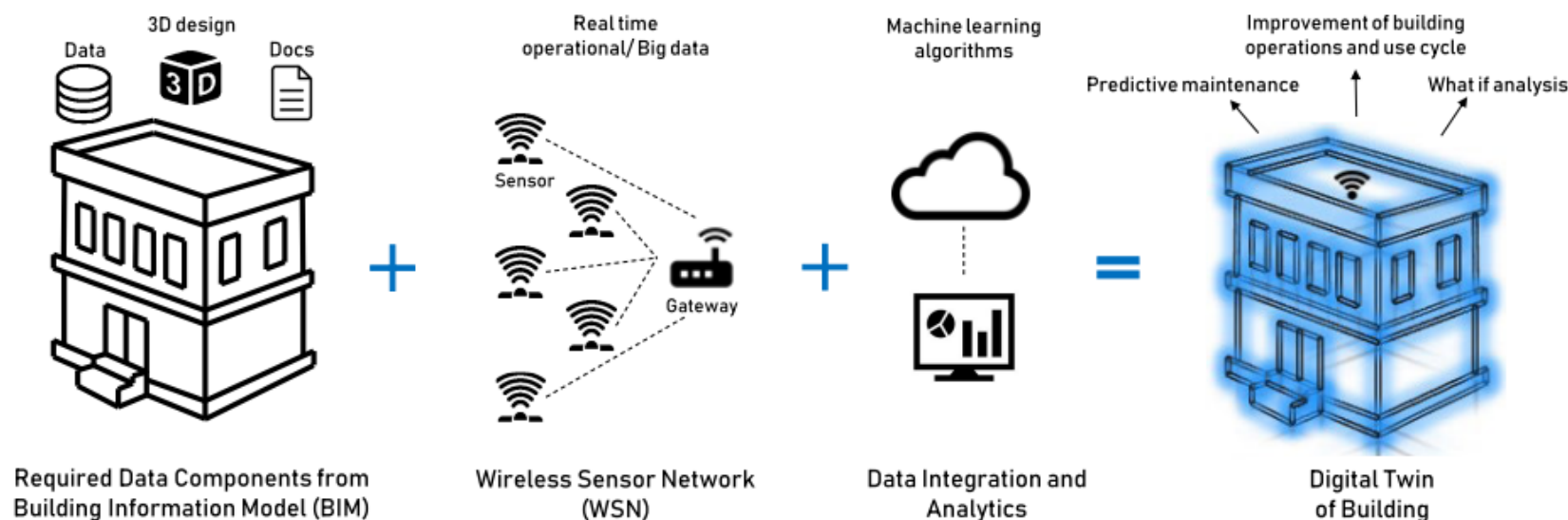
第谷长期积累的精确数据加上开普勒创新的思想，对原始观测数据的尊重 and 有效利用，共同铸就了这一辉煌成就



开普勒行星运动三大定律：轨道定律、面积定律和周期定律

数据的用处： 例子 ②

数字孪生是充分**利用**物理模型、传感器更新、运行历史等**数据**，集成多学科、多物理量、多尺度、多概率的仿真过程，在虚拟空间中完成映射，从而**反映相对应的实体装备的全生命周期过程**



(工业制造 → 军事应用 → 生命体)

S. H. Khajavi, N. H. Motlagh, A. Jaribion, L. C. Werner and J. Holmström, "Digital Twin: Vision, Benefits, Boundaries, and Creation for Buildings," *IEEE Access*, vol. 7, pp. 147406-147419, 2019

数据的用处：例子 ②



2023年，虚拟邓丽君登陆北京春晚



2024年，商汤用AI复活了汤晓鸥



2023年，数字人点燃亚运主火炬

数实结合新时代！



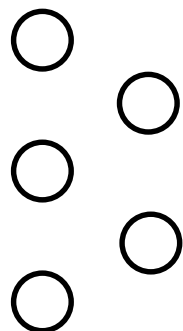
数据的用处： 例子 ③

图灵奖得主吉姆·格雷(Jim Gray) 将科学研究分为四类范式 (Paradigm, 某种必须遵循的规范或大家都在用的套路), 依次为实验归纳, 模型推演, 仿真模拟和数据密集型科学发现 (Data-Intensive Scientific Discovery)。其中, 最后的“数据密集型”, 也就是现在我们所称的“科学大数据”

- 实验科学 (第一范式): 钻木取火、伽利略比萨斜塔实验
- 理论科学 (第二范式): 牛顿三大定律 (经典力学), 麦克斯韦理论 (电磁学)
- 计算机仿真 (第三范式): 模拟核试验、天气预报
- 数据科学 (第四范式): 先有了大量的已知数据, 然后通过计算得出之前未知的可信的理论 (沃尔玛超市-尿布与啤酒的例子)

数据的逻辑结构

- 数据的基本单元称为数据元素，数据元素并不是孤立存在的，而是存在密切的**联系**（→**信息、知识**）
- 数据的逻辑结构（集合上的序关系）：或反映数据本身的特性，或是人们为了处理需要而赋予数据的。

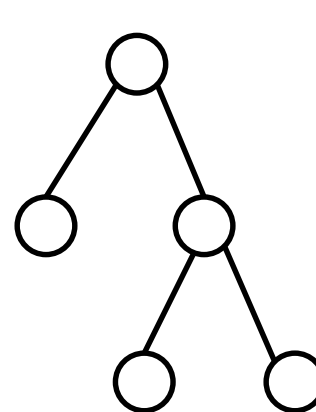


集合



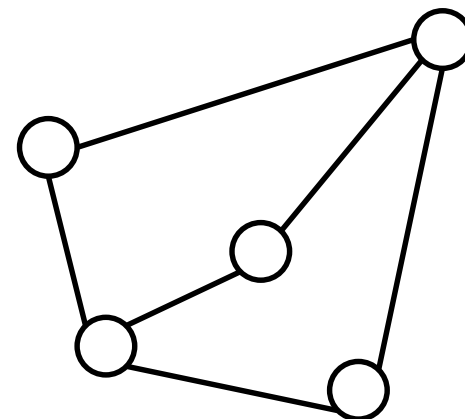
线性结构

线性表、栈、队列、字符串



树结构

树与二叉树



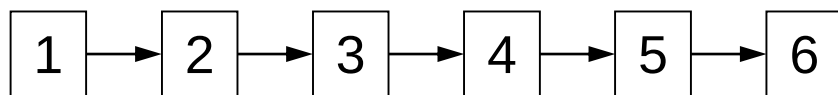
图结构



数据的存储结构

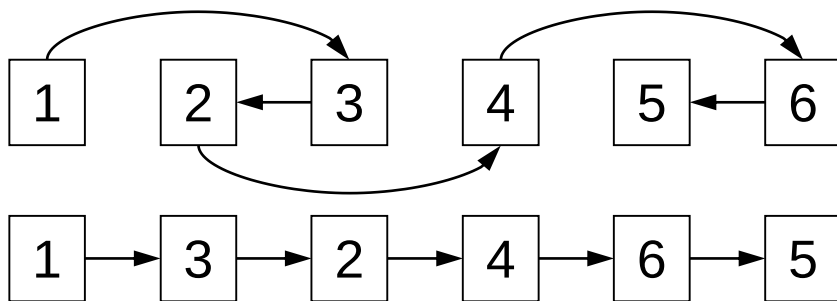
数据的存储结构：数据在计算机中的组织方式

- 顺序存储：以数据元素在存储器中的相对位置表示元素间的逻辑关系



数据元素的存储相对位置和元素间的逻辑关系是一致的

- 链式存储：通过附加指针来表示元素之间的逻辑关系



数据元素的存储相对位置和元素间的逻辑关系没有必然联系，数据元素之间的逻辑关系靠附加指针来维护



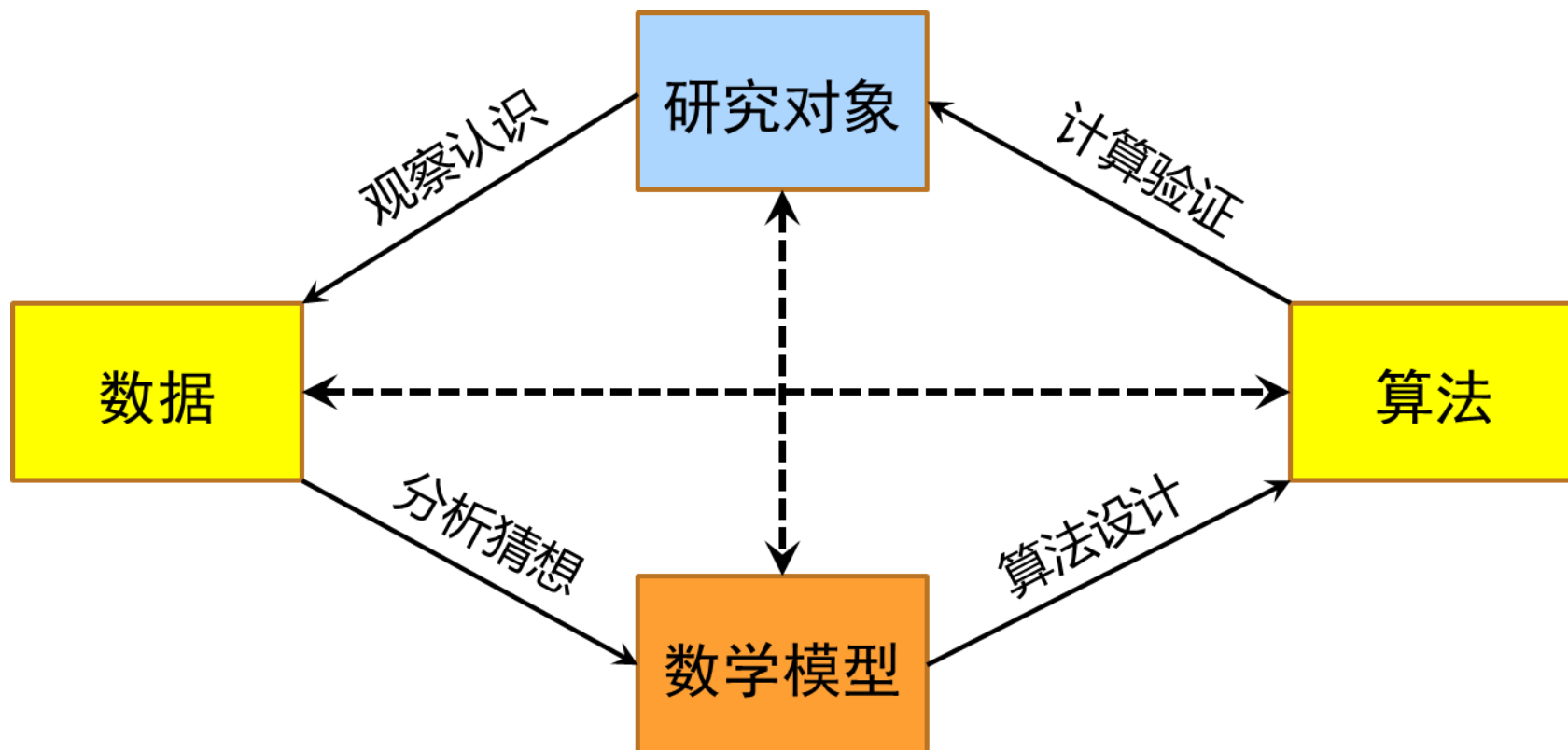
提纲

- 课程概述
- 数据
- 数学模型
- 算法



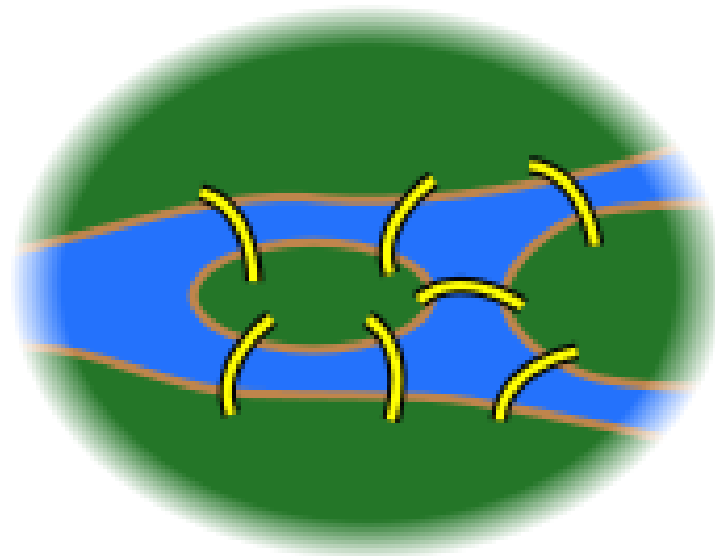
数学模型

数学模型是对于现实世界的某一特定对象，根据其内在规律，为特定目的而得到的一个抽象的、简化的数学结构



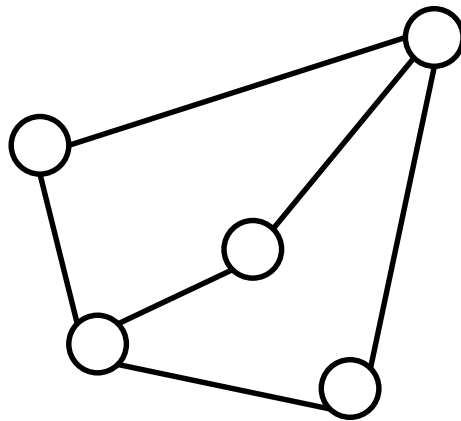
数学模型： 例子 ①

- 10个人参加一个聚会，之前有些人互相认识，有些不认识。组织者统计后，每人在聚会前认识的人数分别为3, 3, 3, 3, 5, 6, 6, 6, 6, 6。请问统计是否有错？
- 一笔画问题：日、中、田、目是否可以一笔画出？
- 科尼斯堡七桥问题：有一个小岛，有七座桥将这个小岛与河的两岸以及交叉口之间的陆地连接起来。有没有一种方法连续地穿过镇上的这七座桥，同时又不重复走过每座桥？



数学模型： 例子 ①

- 10个人参加一个聚会，之前有些人互相认识，有些不认识。组织者统计后，每人在聚会前认识的人数分别为3, 3, 3, 3, 5, 6, 6, 6, 6, 6。请问统计是否有错？

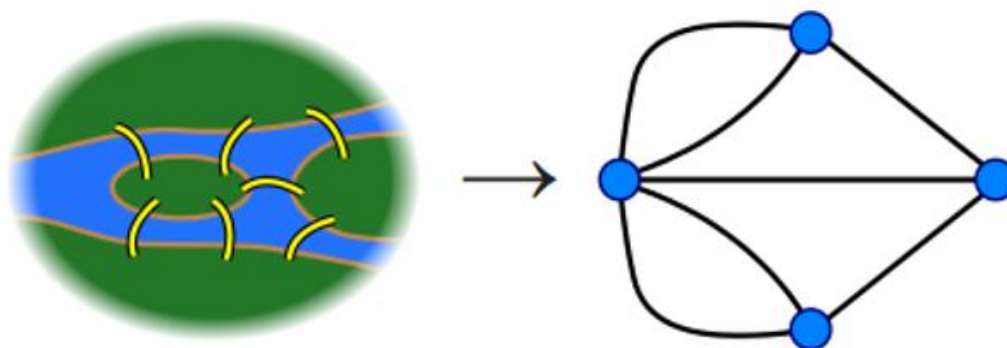


图模型

- 图中一个顶点的度是与它相关联的边的条数
- 握手定理：图中所有顶点的度之和等于边数之和的2倍

数学模型： 例子 ①

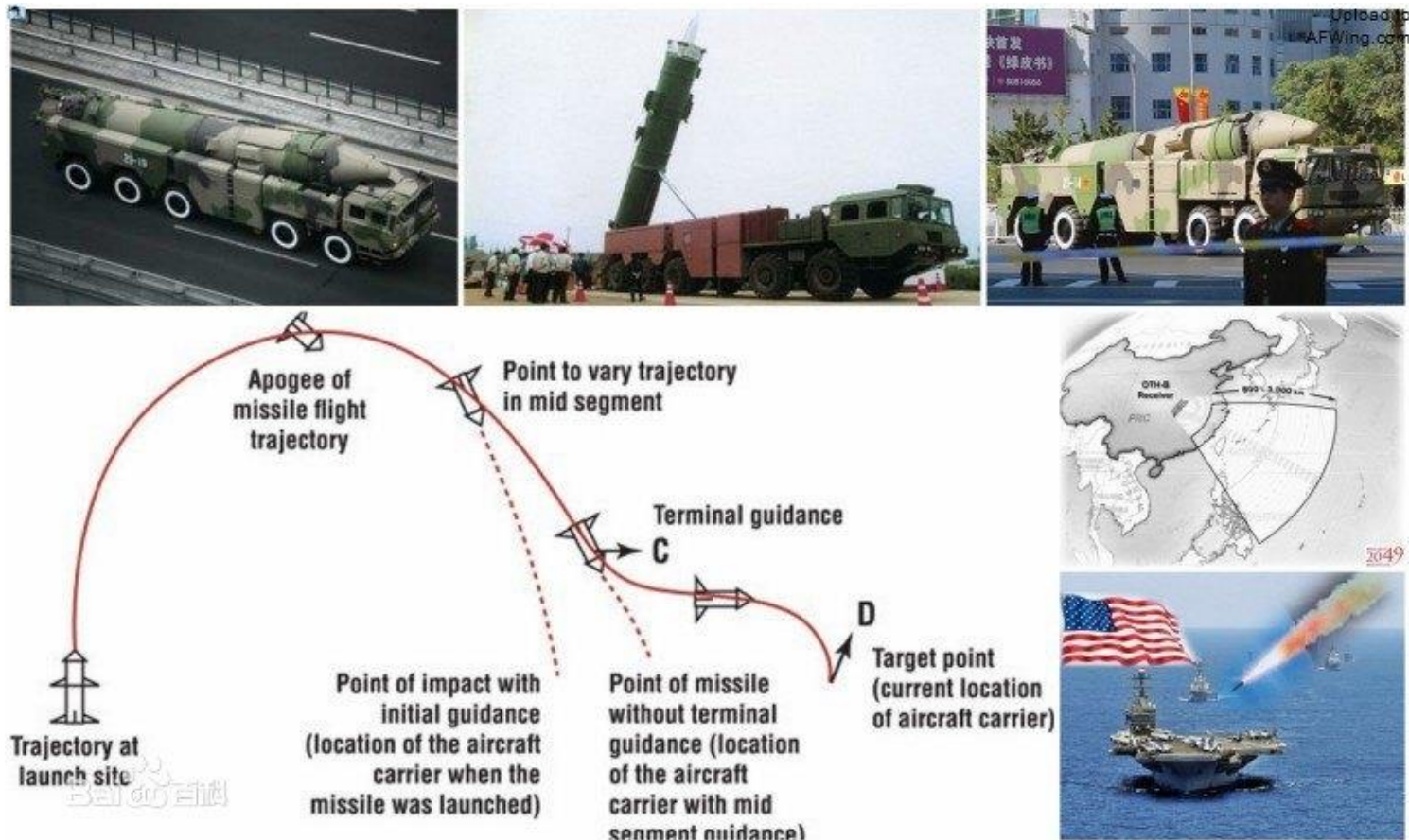
- 一笔画问题：日、中、田、目是否可以一笔画出？
- 科尼斯堡七桥问题



图模型

- 欧拉路径：遍历图中每条边且只访问一次的路径
- 一个图有欧拉路径的充分必要条件：图是连通的且仅有两个奇数度顶点，或者没有顶点的度是奇数

数学模型： 例子 ②



钱学森弹道：将弹道导弹和飞航导弹的轨迹融合在一起，使之既有弹道导弹的突防性能，又有飞航式导弹的灵活性（DF21反舰导弹）——参考百度百科

数学模型： 例子 ②

- 二战期间，美军设立“弹道研究实验室”，为陆军每天提供6张火力表
- 每张表都要计算几百条弹道，而每条弹道的数学模型都是一组非常复杂的**非线性方程组**
- 无法准确求解，只能用数值方法计算 → 计算机诞生（ENIAC, 1946.2.14）
 - 使用了18800个真空管，占地约150平方米
 - 重30吨（大约相当于15-20辆今天的小汽车）
 - 每秒5000次加法运算
 - 通过6000个开关和众多的插头与插座来编程
 - 平均每15分钟有一只真空管损坏
 - 耗电惊人：20万千瓦时





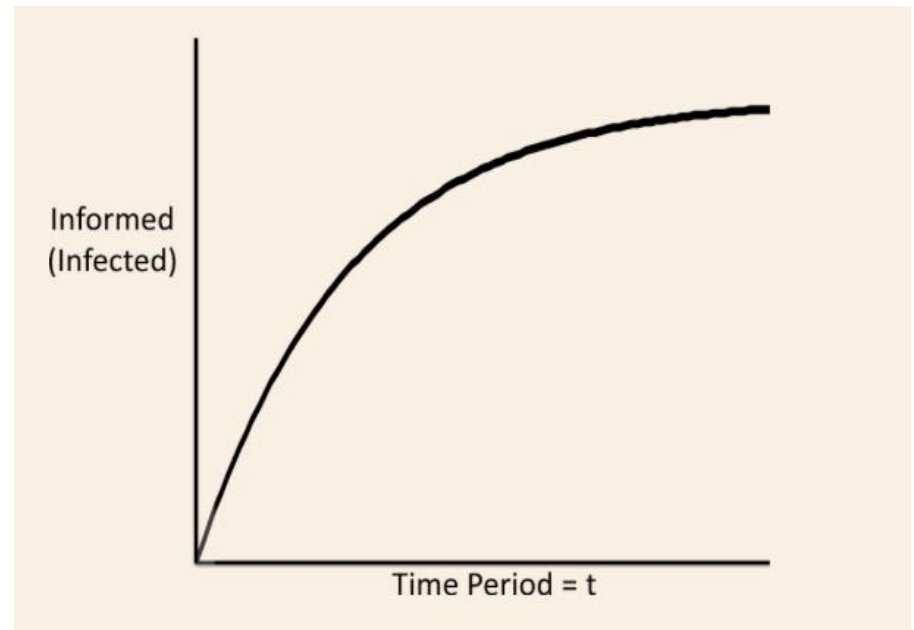
数学模型： 例子 ③

流行病学： 广播模型

广播模型公式：

$$I_{t+1} = I_t + P_b \cdot S_t$$

被感染人数=已感染人数
+感染率×未感染人数



广播的传播曲线

Page S E. The model thinker: what you need to know to make data work for you[M]. Hachette UK, 2018.



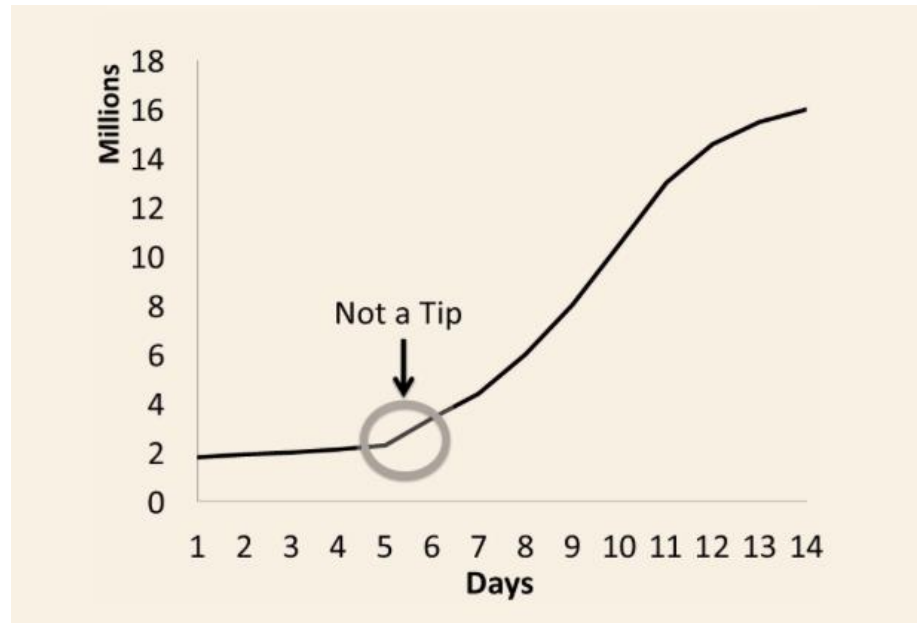
数学模型： 例子 ③

流行病学： 扩散模型

扩散模型公式：

$$I_{t+1} = I_t + P_d \cdot \frac{I_t}{N} \cdot S_t$$

被感染人数=已感染人数+感染率
×未感染人数×已感染人数在全
部相关人群中的比例



扩散的传播曲线

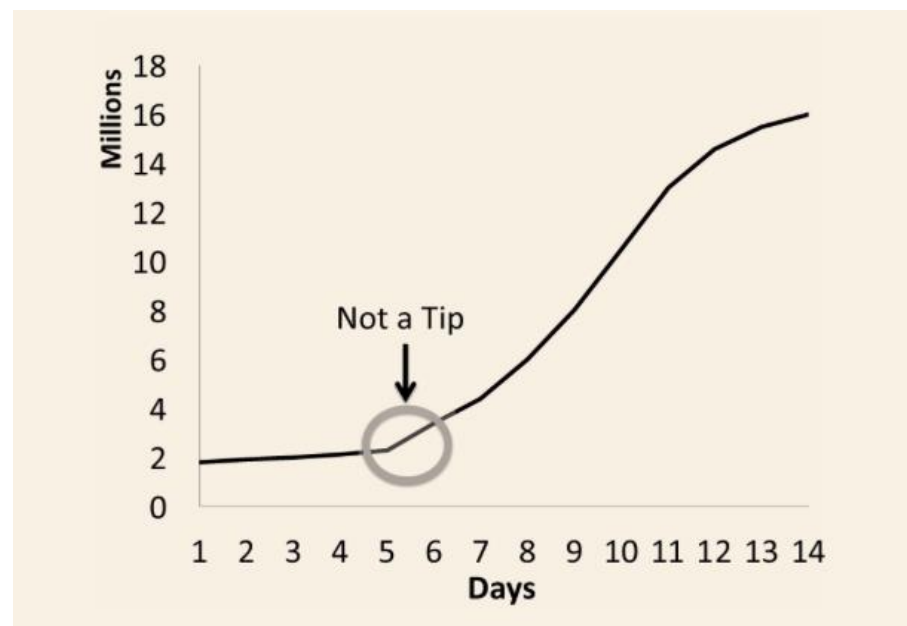
Page S E. The model thinker: what you need to know to make data work for you[M]. Hachette UK, 2018.



数学模型： 例子 ③

很多事情可能都有S曲线的规律，趋势变化往往由于关键影响因素改变

该图反映Google+服务推出后用户的增长情况。Google并没有大搞宣传，主要靠人传人。前4天用户很少，传播速度慢，第5天迎来拐点。等到可能会用的人都用上了，扩散结束



扩散模型的S曲线

Page S E. The model thinker: what you need to know to make data work for you[M]. Hachette UK, 2018.



数学模型： 例子 ③

流行病学： SIR模型

SIR模型公式：

$$I_{t+1} = I_t + P_d \cdot \frac{I_t}{N} \cdot S_t - P_r I_t$$

被感染人数=已感染人数+感染率×未
感染人数×已感染人数在全部相干人
群中的比例-恢复率×已感染人数

另一个关键因素是“恢复”。信息会被人忘记，传染病会痊愈，流行会退潮。如果感染病毒一段时间之后痊愈了，就不再是传染源

SIR模型考虑了恢复率



数学模型： 例子 ③

SIR模型公式：

$$I_{t+1} = I_t + P_d \cdot \frac{I_t}{N} \cdot S_t - P_r I_t$$

疾病预防控制中心关注 R_0 ，称为“基础繁殖数（basic reproduction number）”，是扩散概率和恢复概率之间的比值

$$R_0 = \frac{P_d}{P_r}$$

$R_0 > 1$ ，病毒会传播到整个相关人群

$R_0 < 1$ ，它就不能发挥全部的潜能，会自行消亡

Page S E. The model thinker: what you need to know to make data work for you[M]. Hachette UK, 2018.



数学模型： 例子 ③

新冠r0值是多少

×

📷

百度一下

Q 网页

🖼️ 图片

📰 资讯

📺 视频

📖 笔记

📍 地图

📄 贴吧

📚 文库

更多

百度为您找到以下结果

🔍 搜索工具

🤖 智能回答 >

🔊

2到3

新冠病毒的R0值大约在2到3之间。这个范围是根据不同的研究数据得出的，具体数值可能会因病毒的变异和传播环境的不同而有所变化。 1 2

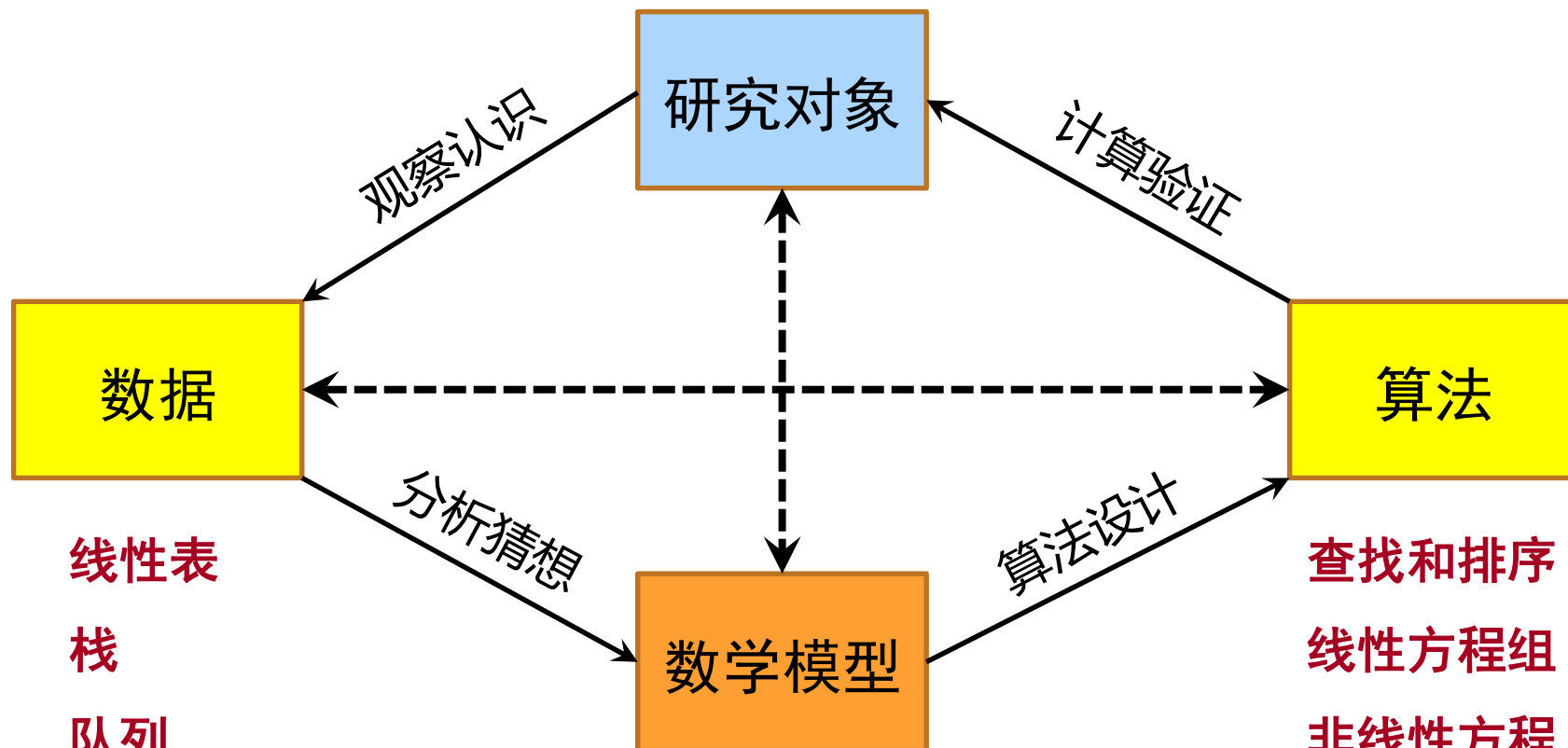
R0值，即基本传染数，是指在没有任何外力介入且所有人没有免疫力的情况下，一个感染者平均能传染给其他多少个人。这个数值受到多种因素的影响，包括感染期的长短、感染者与易感者之间的传播概率以及接触频率等。

R0值的变化对疫情防控有着重要影响。较高的R0值意味着病毒传播速度更快，防控难度更大。因此，通过降低人与人之间的接触频率、佩戴口罩、保持社交距离等措施可以有效降低R0值，从而控制疫情的传播。

可查阅专业文献进一步研究新冠R0值



数学模型



线性表

栈

队列

树与二叉树

图 ...

Model Thinker: 模型思维

→ 算法设计思想

查找和排序

线性方程组

非线性方程

插值与拟合

最优化 ...



提纲

- 课程概述
- 数据
- 数学模型
- 算法

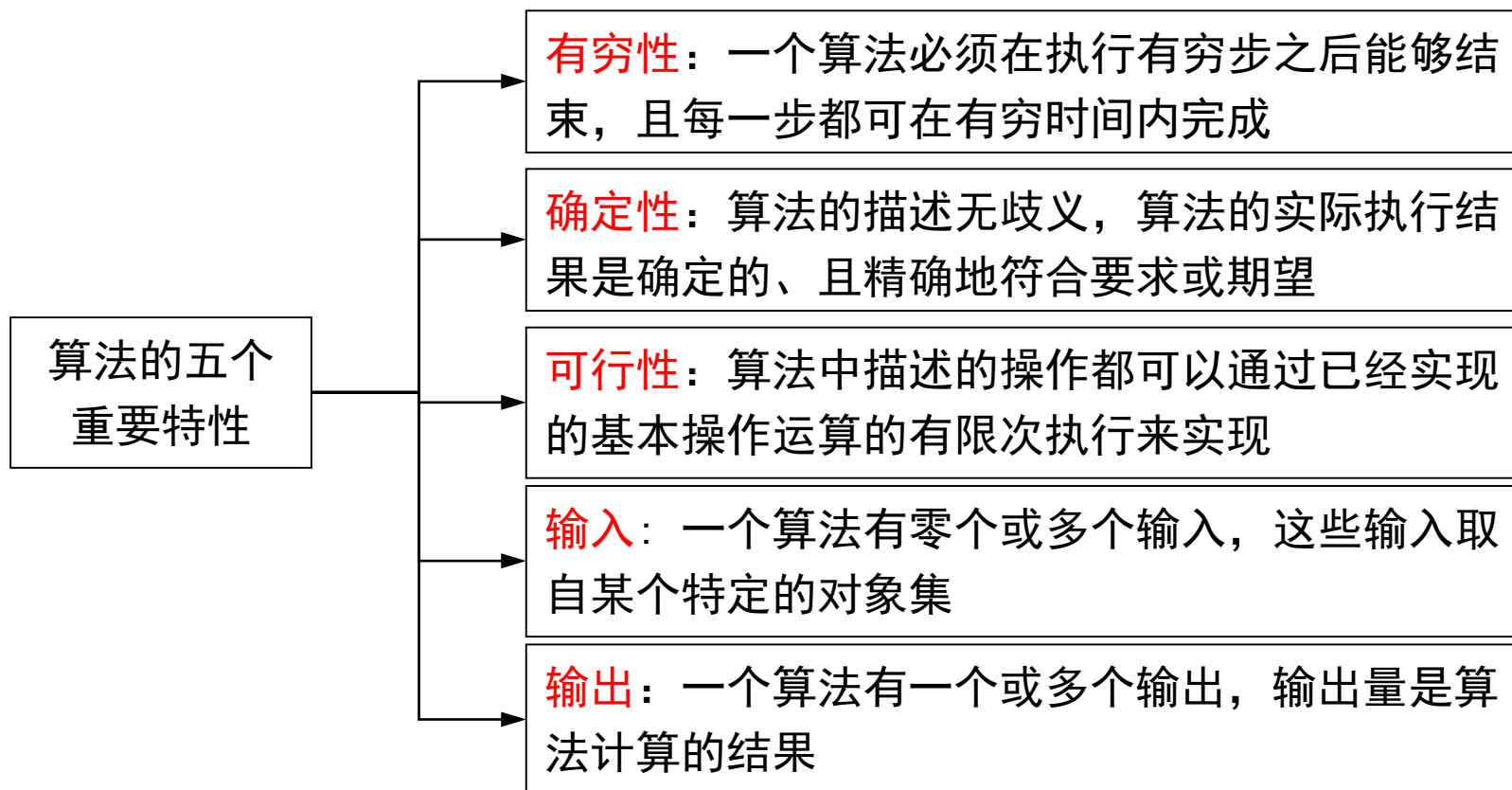


算法

- 算法：解决特定问题的方法
- 算法是求解问题的有限长度的指令序列，是特定问题的程序化解决方案
- 计算机算法只是算法的一个子集
 - 在计算机发明之前人们已经开始使用算法，如辗转相除法和《九章算术》（公元一世纪）第八章“方程”提出的线性方程组求解算法
 - 算法也不局限与数学，如按照菜谱做菜
- 关注能够输入到计算机并被计算机处理（计算机算法）的数据



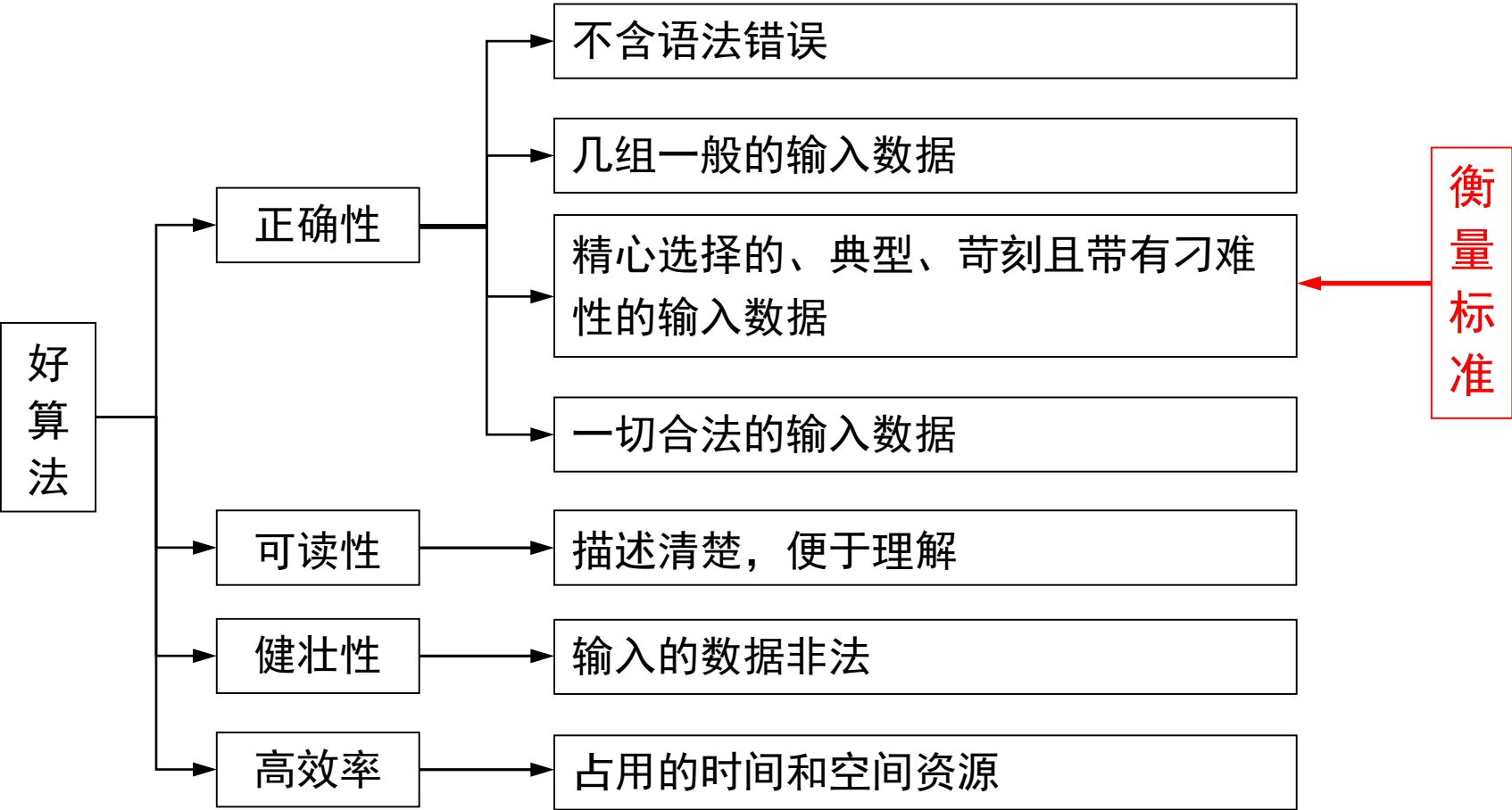
算法的特征



Donald E. Knuth, The Art of Computer Programming, Addison-Wesley, 2011



算法的评价





算法效率举例

尽管计算机的计算和存储能力增长迅速，算法的作用仍然不是计算机硬件性能提升所能替代的，算法效率仍然非常重要

排序方法	时间复杂度	系数取值	运行速度
插入排序	C_1n^2	2	计算机A: 10^9
归并排序	$C_2n\log_2n$	50	计算机B: 10^7

序列规模 $n=10^6$		序列规模 $n=10^7$	
指令数	运行时间	指令数	运行时间
$2*10^{12}$	2000s	$2*10^{14}$	2.3day
10^9	100s	$1.17*10^{10}$	20min



算法的时间复杂度

■ 事后统计法：

- 必须实现算法并且执行程序
- 代价可能较大（资源、时间）
- 很多因素影响运行时间，掩盖算法本质，影响分析

■ 事前分析估算法



算法的时间复杂度

- 和算法执行时间相关的因素：
 - 数据结构和算法策略的选择
 - 问题的规模
 - 编写程序的语言（高级语言不如汇编语言）
 - 编译程序产生的机器代码的质量（编译器影响）
 - 计算机速度、当前运行情况
- 一个特定算法的运行工作量的大小，是问题规模的函数
(不考虑软硬件平台的影响→通用性和可移植性)



算法的时间复杂度

- 算法的渐进时间复杂度 (Time Complexity) : $T(n) = O[f(n)]$
 - 关注的是趋势，也就是问题规模足够大以后，算法时间复杂度所遵循的规律
- **Big-O** 记号的形式化定义：
 - 若 $f(n)$ 是正整数 n 的一个函数，则 $x_n = O[f(n)]$ 表示存在正的常数 M 和 n_0 ，使得当 $n > n_0$ 时，都满足 $|x_n| \leq M|f(n)|$
 - 标记的是算法效率的上限
 - 如果 c 是一个正常数， $O[cf(n)] = O[f(n)]$
 - 如果 $g(n)$ 的阶数小于等于 $f(n)$ 的阶数，有 $O[f(n) + g(n)] = O[f(n)]$



算法的时间复杂度

- 算法的执行时间= $\sum$ 操作的执行次数 $\times$ 操作的执行时间
- 算法操作包括控制操作和原操作
 - 控制操作决定了原操作的执行次数，本身复杂度可以忽略
 - 原操作中执行次数最多的一种或几种操作称为基本操作
- 选取算法中的基本操作
- 算法的执行时间与基本操作执行次数之和成正比



算法的时间复杂度：例 ①

■ 程序片段

```
for( i = 0; i < n; i++) {  
    for(j = 0; j < n; j++) {  
        c[i][j] = 0;  
        for( k = 0; k < n; k++)  
            c[i][j] +=  
                a[i][k] * b[k][j];  
    }  
}
```

■ 时间复杂度

三层嵌套循环，最内层的循环体的时间复杂度为 $O(1)$ ，三层嵌套循环的执行次数都是 n 次，因此整个程序片断的时间复杂度为：
$$n \times n \times n \times O(1) = O(n^3)$$



算法的时间复杂度：例 ②

■ 程序片段

```
for(i = 0; i < n; i ++){  
    for(j = i; j < n; j++){  
        if(A[i] > A[j]) {  
            temp = A[i];  
            A[i] = A[j];  
            A[j] = temp;  
        }  
    }  
}
```

■ 时间复杂度

两层嵌套循环，内层 j 的循环从当前 i 的值开始，因此if语句体的执行次数为 $n + (n - 1) + (n - 2) + \dots + 2 + 1$ ；因此整个程序片段的时间复杂度为：

$$\begin{aligned} \sum_{i=1}^n i \times O(1) &= O(1) \times \sum_{i=1}^n i \\ &= \frac{n(n+1)}{2} \times O(1) \\ &= O\left(\frac{n^2+n}{2}\right) = O\left(\frac{n^2}{2}\right) = O(n^2) \end{aligned}$$



算法的时间复杂度：例 ③

■ 程序片段

```
for( i = 0; i < n; i++){  
    for(j = 1; j < i; j*=2) {  
        A[i] += A[j];  
    }  
}
```

■ 时间复杂度

两层嵌套循环，内层的循环体的时间复杂度为 $O(1)$ ，由于内层循环的增量为 $2 \times j$ ，结束条件为 $j < i$ ，因此内层循环的时间复杂度为 $O(\log_2 i)$ ，因此整个程序片断的时间复杂度为：

$$\sum_{i=2}^{n-1} O(\log_2 i) = O\left(\sum_{i=2}^{n-1} \log_2 i\right) = O\left(\log_2 1 + \sum_{i=2}^{n-1} \log_2 i + \log_2 n\right)$$

$$= O\left(\log_2 \prod_{i=1}^n i\right) = O(\log_2(n!)) = O(\log(n!)) = O(n \log(n)) \quad n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$



算法的时间复杂度：例 ④

■ 程序片段

```
for(i = 1; i < n; i*=2) {  
    j = 0;  
    while(j < 10) {  
        A[i] += A[j];  
        j++;  
    }  
}
```

■ 时间复杂度

两层嵌套循环，内层的循环体的时间复杂度为 $O(1)$ ，由于内层while循环执行次数为常量10，外层for循环的执行次数为 $\log_2 n$ ，因此整个程序片断的时间复杂度为：

$$\log_2 n \times 10 \times O(1) = O(\log_2 n)$$



算法的时间复杂度

■ 描述指标

- 最好情况 (best-case)：对于任何一个输入的运行时间下限
- 最坏情况 (worst-case)：对于任何一个输入的运行时间上限
- 平均情况 (average-case)：对大量输入的平均运行时间

■ 最重要的是平均情况下的性能

■ 分析举例

- 在元素集合查找元素



算法的时间复杂度

在元素集合中查找元素

- 逐个比较，确定是否是要查找元素；
- 最优情况：第一个元素就是要查找元素；
- 最坏情况：最后一个元素才是要查找的元素
- 平均情况：每个进行比较的元素都可能是要查找的元素，
 - 概率相等： $p(i) = p = 1/n$
 - 平均比较次数

$$T(n) = \sum_{i=1}^n p(i) \times i = p \times \sum_{i=1}^n i = p \times \frac{n(n+1)}{2} = (n+1)/2$$



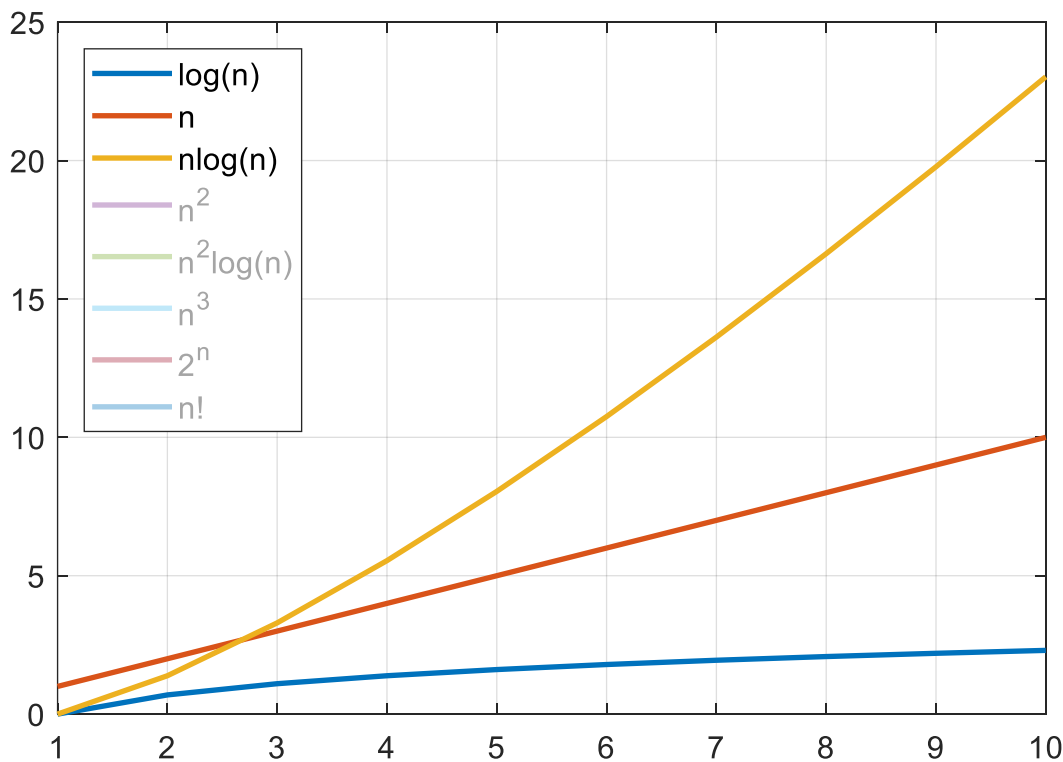
算法的空间复杂度

- 算法的存储和运行还需要占用存储器
 - 指令空间(instruction space): 是用来存储程序指令所需空间
 - 数据空间(data space): 存储运行过程中常量和变量所需的空間
 - 环境空间: 操作系统为程序运行, 特别是函数调用提供的空间
- 程序指令是对程序代码进行编译得到的机器语言指令, 一般占据空间规模较小; 而输入数据所占空间只取决于问题本身, 和算法无关; 因此, 一般只需要分析除输入和程序之外的额外空间
- 与时间复杂度类似, 算法的渐进空间复杂度: $S(n)=O[f(n)]$



常见复杂度对比

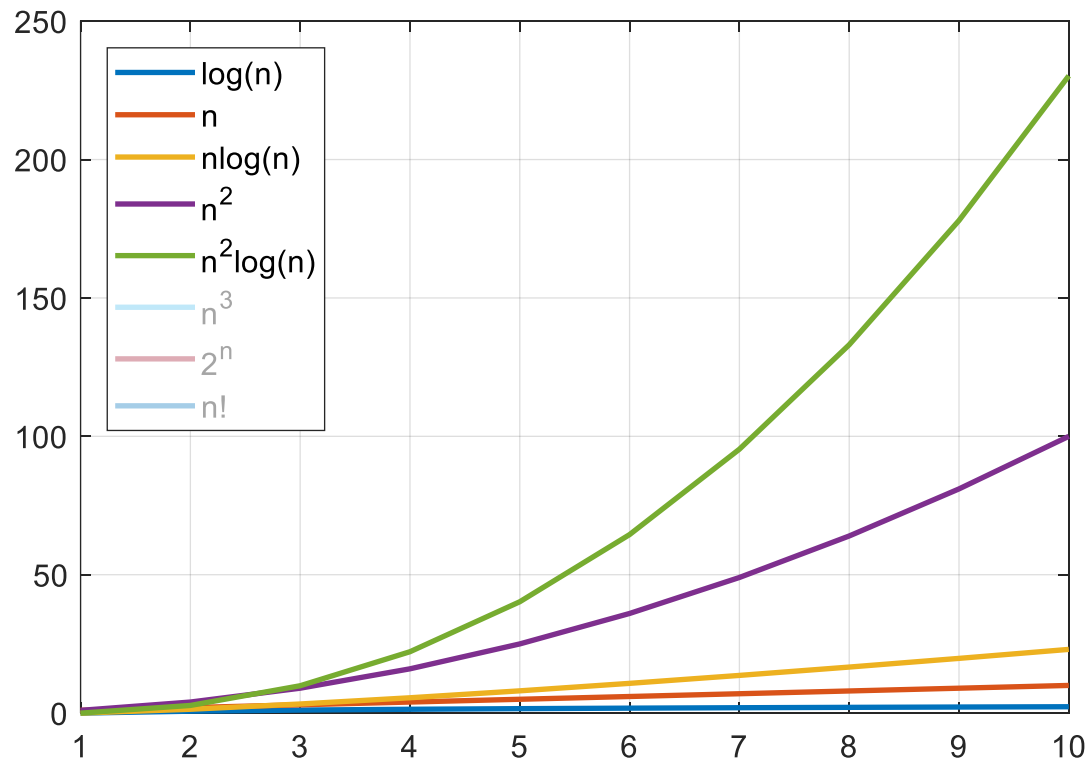
1 , $\log(n)$, n , $n\log(n)$, n^2 , $n^2\log n$, n^3 , 2^n , $n!$





常见复杂度对比

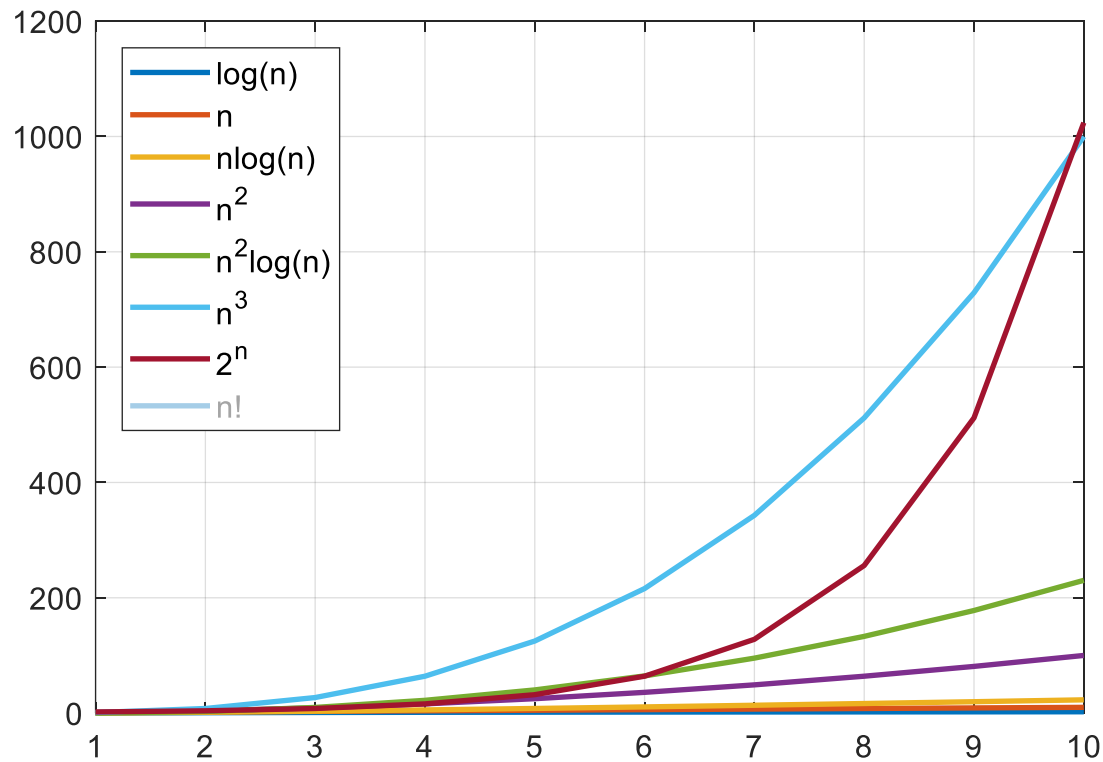
1 , $\log(n)$, n , $n\log(n)$, n^2 , $n^2\log n$, n^3 , 2^n , $n!$





常见复杂度对比

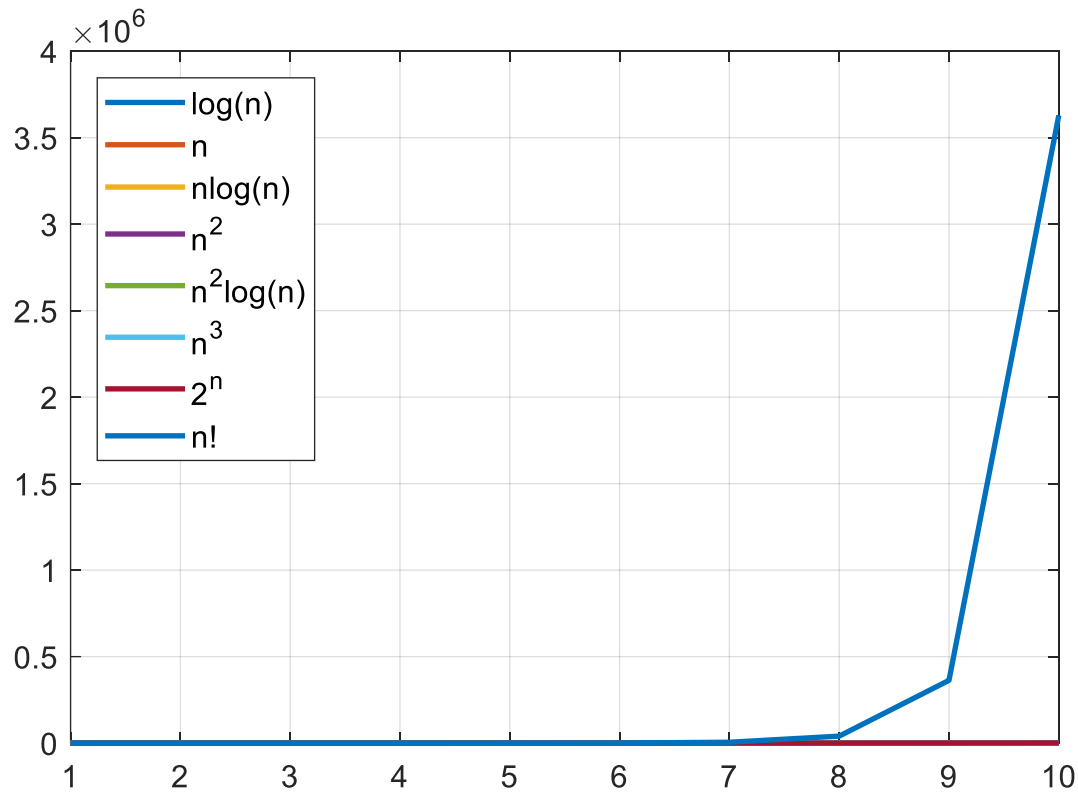
1 , $\log(n)$, n , $n\log(n)$, n^2 , $n^2\log n$, n^3 , 2^n , $n!$





常见复杂度对比

1 , $\log(n)$, n , $n\log(n)$, n^2 , $n^2\log n$, n^3 , 2^n , $n!$





算法效率小结

- 算法效率分析包含时间复杂度和空间复杂度
- 引入 $Big - O$ 表示渐进时间复杂度和空间复杂度
- 迅速找到会被多次反复执行的基本操作
- 常见复杂度形式

$1, \log(n), n, n\log(n), n^2, n^2 \log n, n^3, 2^n, n!$



改进算法效率的意义

网易首页 > 网易号 > 正文

[申请入驻 >](#)

MIT新研究：43%算法对性能的改进超越摩尔定律

2021-09-22 15:22:31 来源: 量子位

[举报](#)



56

分享至



博雯 发自 凹非寺

量子位 报道 | 公众号 QbitAI

软件算法对计算速度的提升有多大？

MIT最新研究说：超过4成算法对性能的改进，已经超过了硬件的摩尔定律。

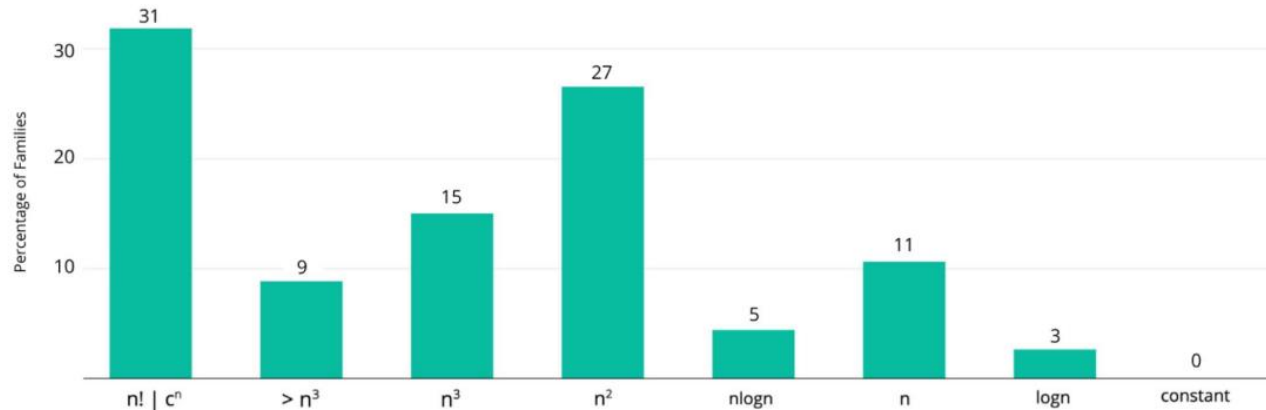
对于中等规模的问题，30%-43%的算法的改进比硬件进步更能提升性能。

当问题数据增加到数亿规模时，算法改进变得比硬件改进/摩尔定律更重要。

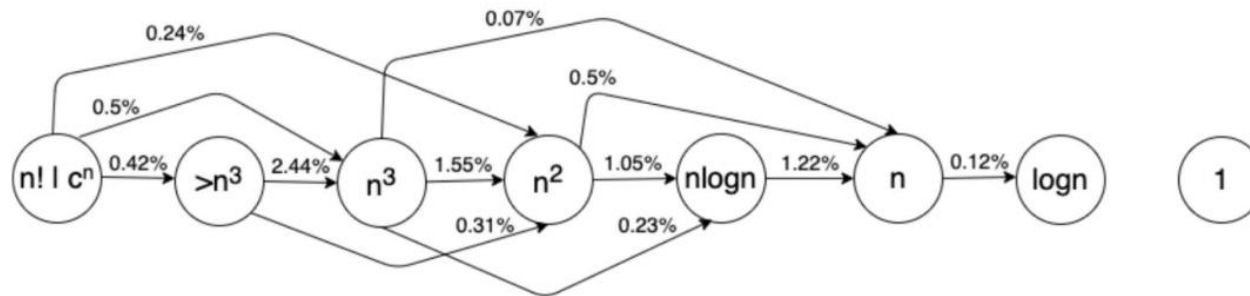
这就是MIT的两位科学家对来自57本教科书，超过1137篇研究论文的数据进行分析后得到的结论。

摩尔定律：集成电路上可以容纳的晶体管数目大约每经过18个月到24个月便会增加一倍，换言之，处理器的性能每隔两年翻一倍。

改进算法效率的意义



(c)



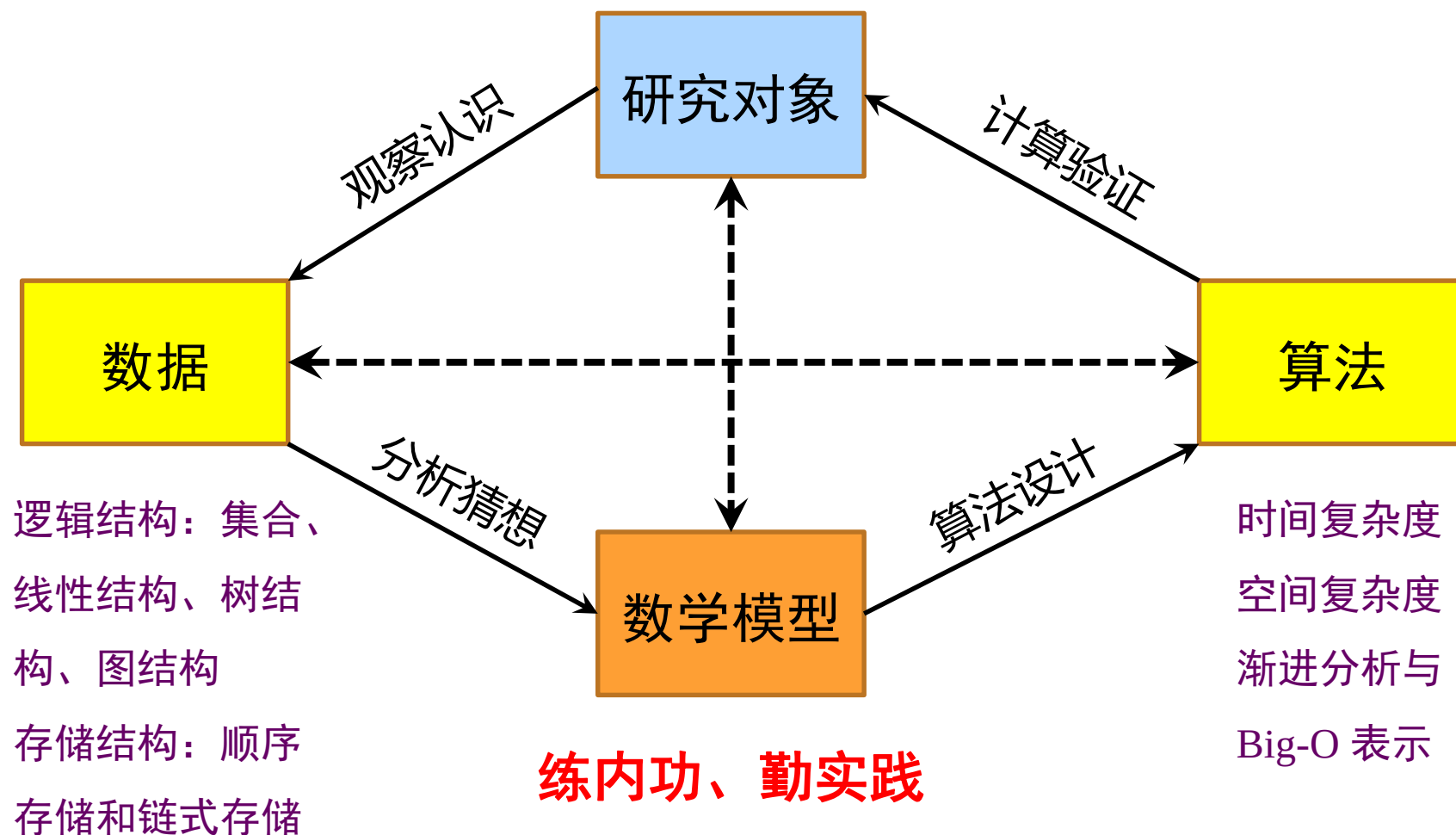
(d)

Fig. 1. Algorithm discovery and improvement. (a) Number of new algorithm families discovered each decade. (b) Share of known algorithm families improved each decade. (c) Asymptotic time complexity class of algorithm families at first discovery. (d) Average yearly probability that an algorithm in one time complexity class transitions to another (average family complexity improvement). In (c) and (d) “ $>n^3$ ” includes time complexities that are superpolynomial but subexponential.

Y. Sherry and N. C. Thompson, "How Fast Do Algorithms Improve?," Proceedings of the IEEE, doi: 10.1109/JPROC.2021.3107219.



本讲总结



加油同学们!

